

Shiv Nadar University Chennai

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

Experiment 1 Implementation of a Single Layer Perceptron for Binary Classification

1. Objective

The objective of this experiment is to understand the concept of an artificial neuron and implement a Single Layer Perceptron from scratch for binary classification. Students will learn the perceptron learning algorithm, understand the importance of activation functions, visualize the learning process, and evaluate the classifier using a real-world dataset.

2. Background Theory

An Artificial Neuron is the fundamental building block of a neural network. It receives one or more input features, multiplies them by corresponding weights, adds a bias term, and applies an activation function to determine the output. The Single Layer Perceptron is the earliest supervised learning algorithm capable of solving linearly separable binary classification problems.

Mathematical Formulation

Weighted Sum

$$z = \mathbf{w}^T \mathbf{x} + b$$

where

- $\mathbf{x}$: Input vector
- $\mathbf{w}$: Weight vector
- b : Bias
- z : Linear combination

Prediction

$$\hat{y} = f(z)$$

Weight Update Rule

$$\mathbf{w}_{new} = \mathbf{w}_{old} + \eta(y - \hat{y})\mathbf{x}$$

Bias Update

$$b_{new} = b_{old} + \eta(y - \hat{y})$$

where η denotes the learning rate.

3. Activation Functions

An activation function determines the output of a neuron based on the weighted sum of its inputs. It introduces non-linearity, enabling neural networks to learn complex patterns. Although the perceptron uses only the Step Activation Function, modern deep learning models employ differentiable activation functions for efficient optimization through backpropagation.

Step Activation Function

$$f(z) = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases}$$

The Step Activation Function produces a binary output and is therefore suitable only for linearly separable binary classification problems.

Activation	Output Range	Typical Applications
Step	$\{0, 1\}$	Classical Perceptron
Sigmoid	$(0, 1)$	Binary Classification Output Layer
Tanh	$(-1, 1)$	Hidden Layers (Traditional Networks)
ReLU	$[0, \infty)$	Hidden Layers in CNNs and MLPs
Leaky ReLU	$(-\infty, \infty)$	Avoiding Dying ReLU
Softmax	$(0, 1)$	Multi-class Classification Output Layer

Note: This experiment uses only the Step Activation Function. The remaining activation functions will be studied in subsequent experiments involving Multilayer Perceptrons and Deep Neural Networks.

4. Dataset Description

Dataset: Banknote Authentication Dataset **Source:** UCI Machine Learning Repository **Download Link:** <https://archive.ics.uci.edu/dataset/267/banknote+authentication>

Instances	1372
Features	4 Numerical Features
Classes	2
Missing Values	None
Task	Binary Classification

Feature Description

- Variance

- Skewness
- Curtosis
- Entropy

Target Class

- 0 – Authentic Banknote
- 1 – Forged Banknote

6. Experimental Procedure

Task 1: Dataset Exploration

- Load the dataset.
- Display the first five samples.
- Determine dataset dimensions.
- Identify missing values.
- Display descriptive statistics.

Expected Output

Dataset summary and statistical information.

Task 2: Exploratory Data Analysis

Generate

- Histograms
- Correlation Heatmap
- Scatter Plot
- Boxplots

Task 3: Data Preprocessing

- Normalize all numerical features.
- Split the dataset into Training (80%) and Testing (20%).

Task 4: Perceptron Implementation

Implement from scratch

- Weight Initialization
- Bias Initialization

- Step Activation Function
- Forward Propagation
- Perceptron Learning Rule

Task 5: Model Training Display

- Epoch Number
- Misclassified Samples
- Updated Weights
- Updated Bias

Task 6: Model Evaluation Compute

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix

Source Code

https://colab.research.google.com/drive/1PCnzByIxlNJ_Lig3zXdE7kVYDm61d1dd?usp=sharing
<https://github.com/likithainnamuri7/Likitha-Innamuri>

11. Experimental Results

Task 1: Dataset Exploration

- Dataset Size: 1372×5
- Number of Features: 4
- Number of Classes: 2
- Missing Values: 0
- Dataset was complete and required no data cleaning.

Task 2: Exploratory Data Analysis (EDA)

- Feature distributions are non-uniform.

- Variance is the most discriminative feature.
- Scatter plot shows good separation between classes.
- Few outliers are present but do not significantly affect classification.

Task 3: Data Preprocessing

- Training Samples: 1097
- Testing Samples: 275
- Training Feature Shape: (1097, 4)
- Testing Feature Shape: (275, 4)
- All features were successfully normalized before training.

Task 4: Perceptron Implementation

- Learning Rate: 0.01
- Epochs: 100
- Initial Misclassified Samples: 47
- Final Misclassified Samples: 12
- Final Bias: -0.13

Task 5: Model Training

- Training error reduced from 47 to 12 misclassified samples.
- Model parameters converged after several epochs.
- Weight values stabilized during training.
- Bias converged to approximately -0.13 .
- Decision boundary successfully separated most training samples.
- Different learning rates produced similar classification performance.

Task 6: Model Evaluation

Metric	Value
Accuracy	98.55%
Precision	98.43%
Recall	98.43%
F1-score	98.43%

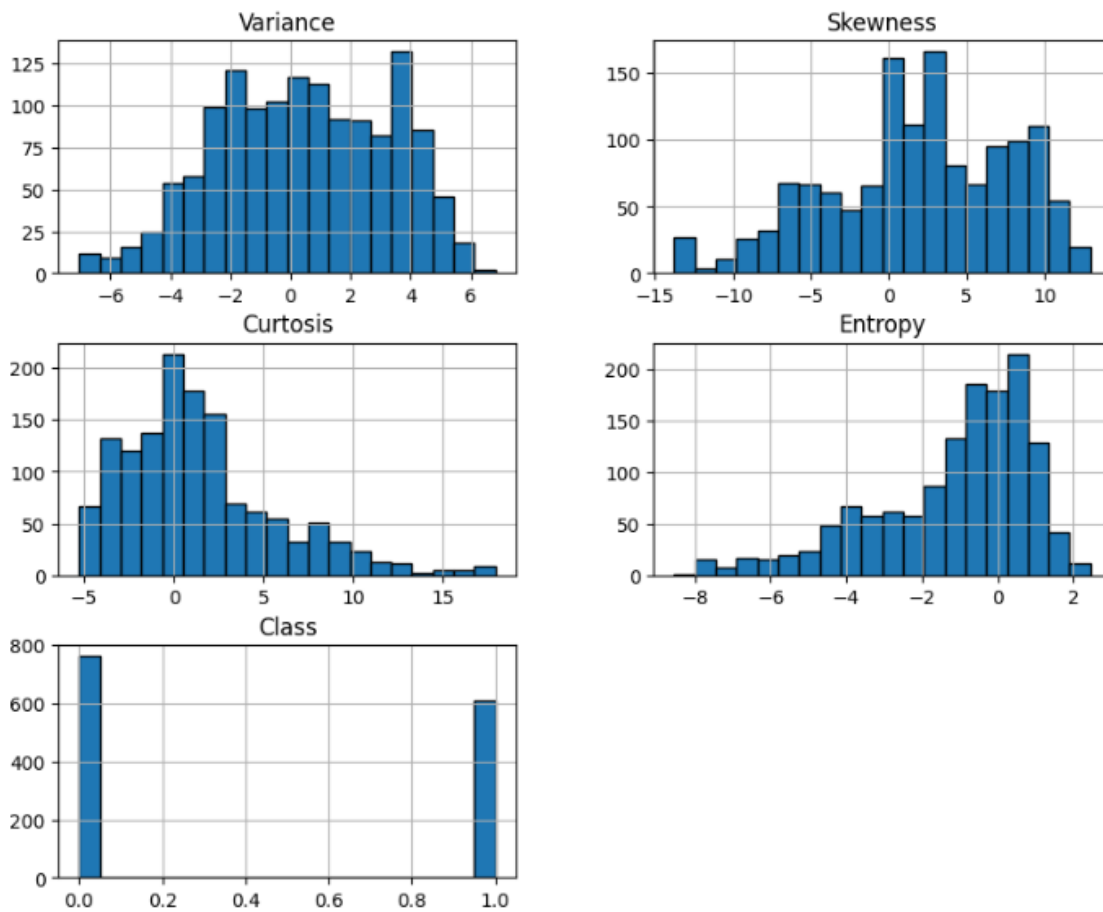
Confusion Matrix

Actual / Predicted	Class 0	Class 1
Class 0	146	2
Class 1	2	125

1 Generated Plots with Interpretation

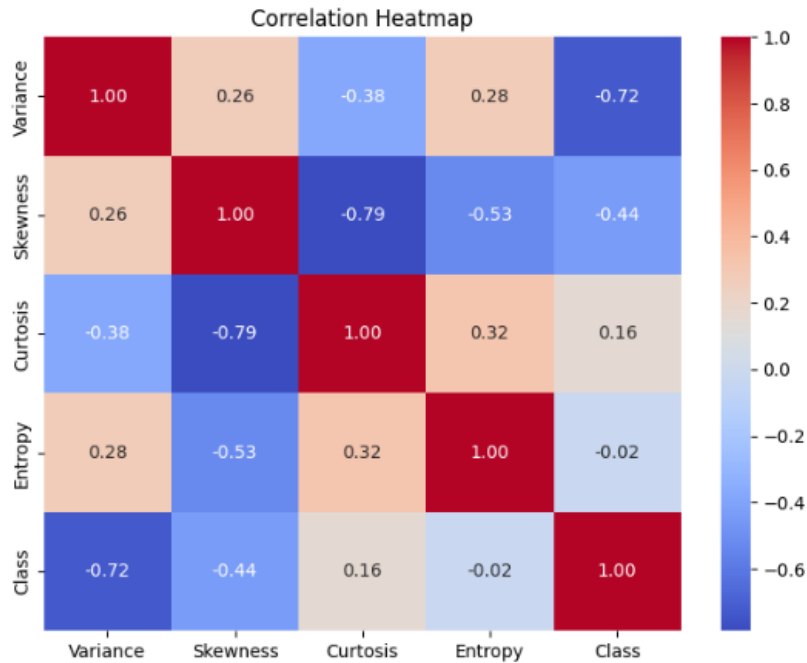
1. Feature Histograms

Feature Histograms



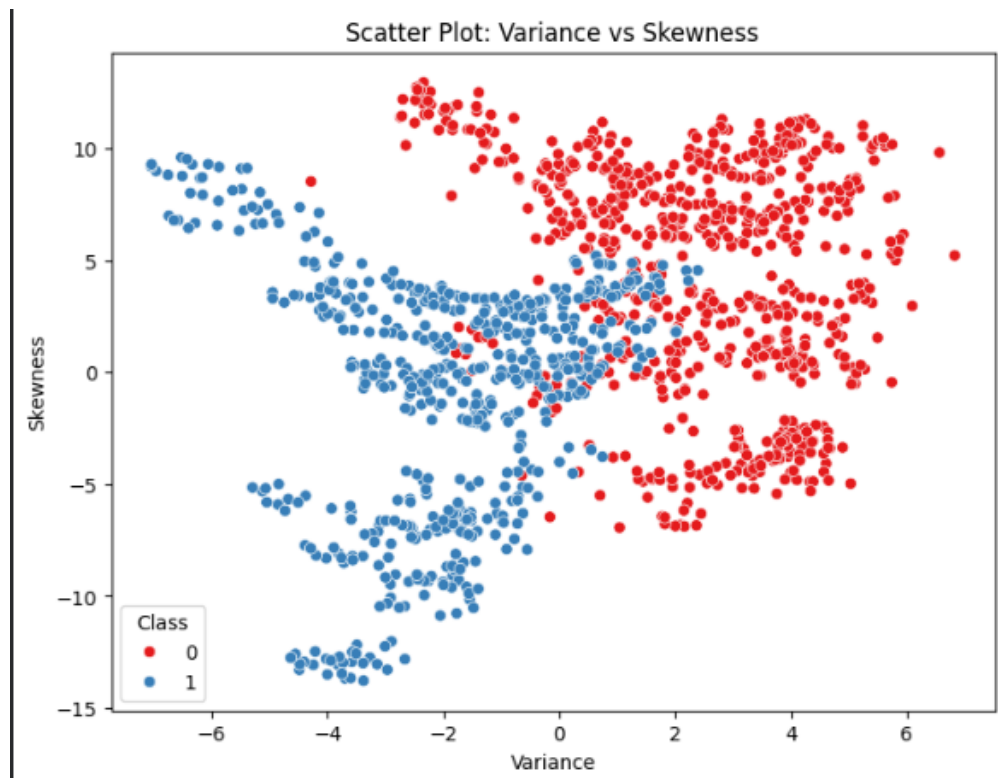
Histograms show that the features that are having different distributions and they are not normally distributed. Kurtosis contains the few extreme values, while the class distribution is nearly balanced between the two categories.

2. Correlation Heatmap



Heatmap indicates that the variance has the strongest correlation with the target class, making it the most influential features. Strong correlation is also observed between skewness and kurtosis

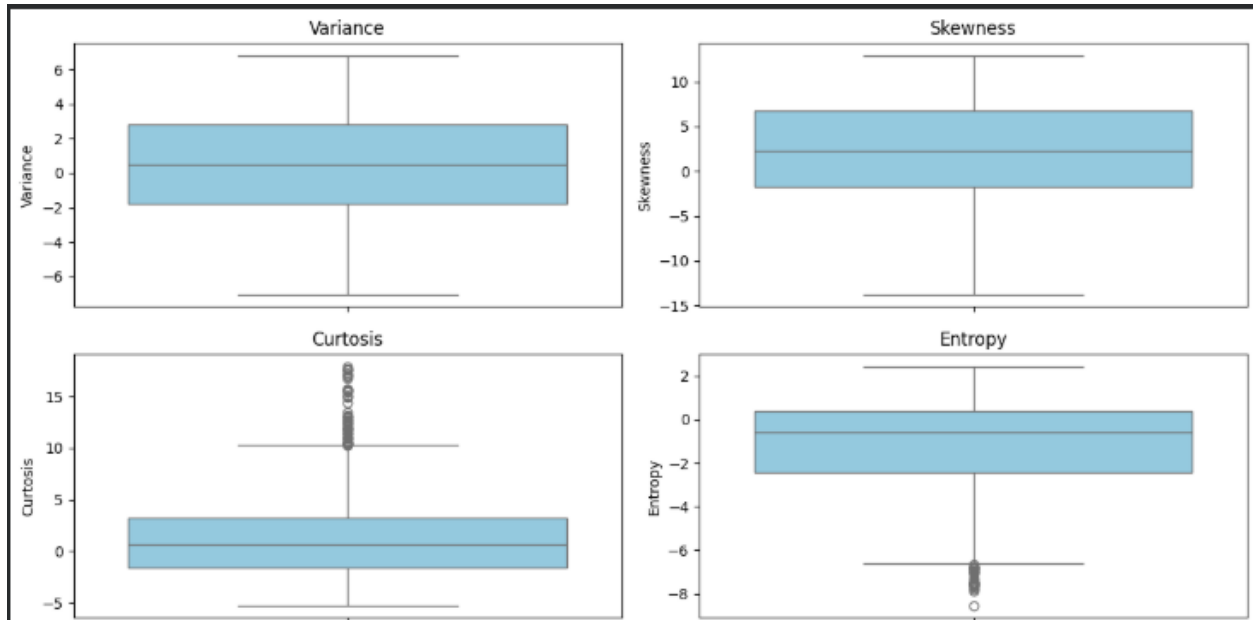
3. Scatter Plot



The scatterplot shows that the two classes that are the well separated with only slight over-

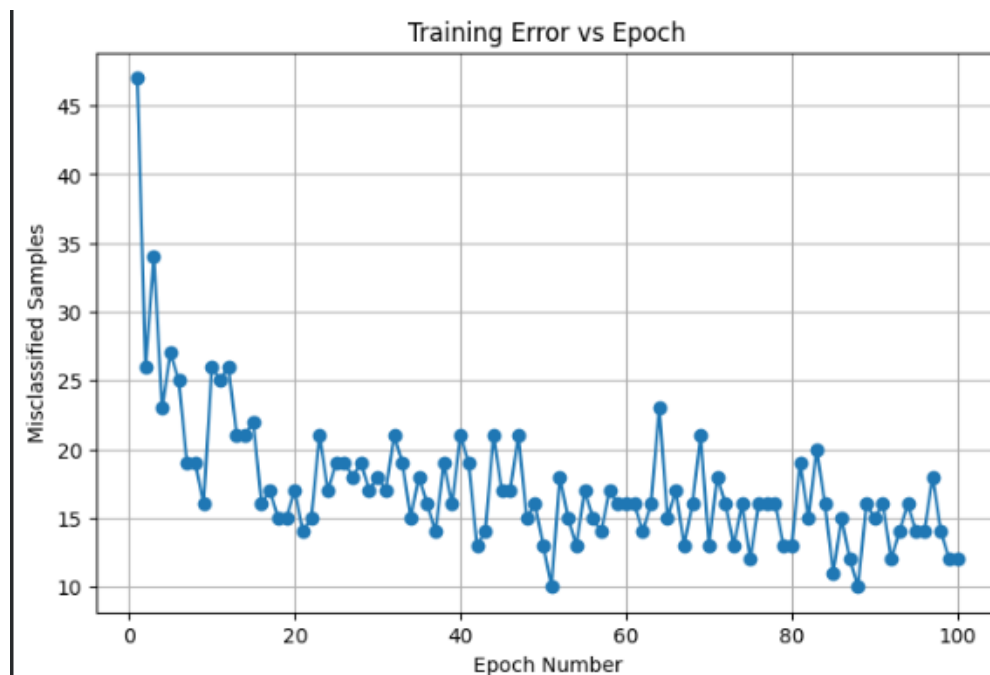
lap.This indicates that the datasets are approximately separable and suitable for the perceptron classification.

4. Boxplots



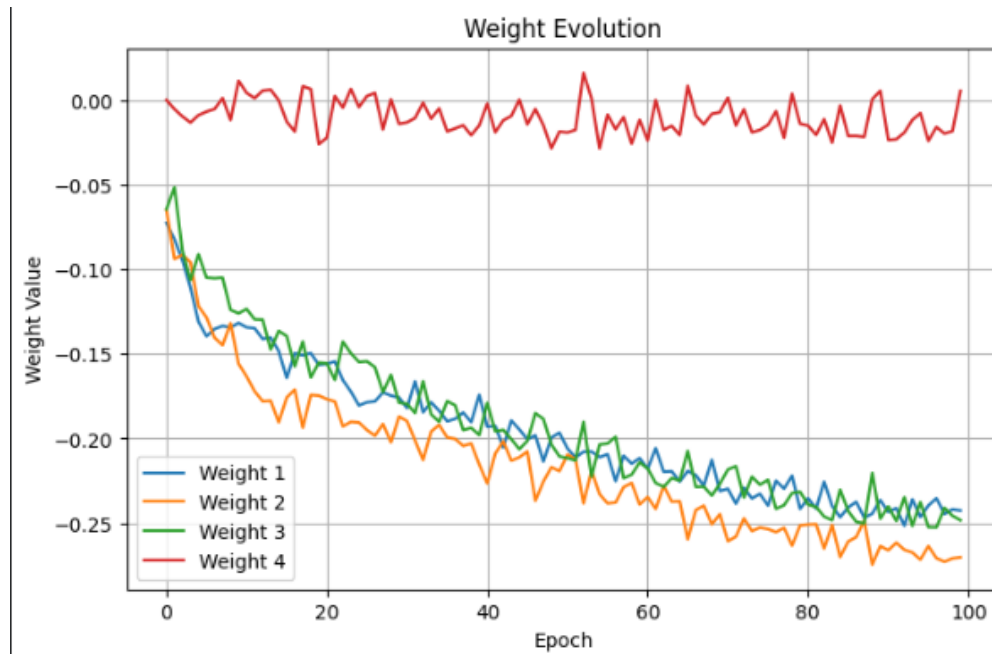
The boxplots indicates kurtosis has several outliers,whereas the remaining features have comparatively less extreme values.These outliers are present as they represent valid observations.

5. Training Error vs Epoch



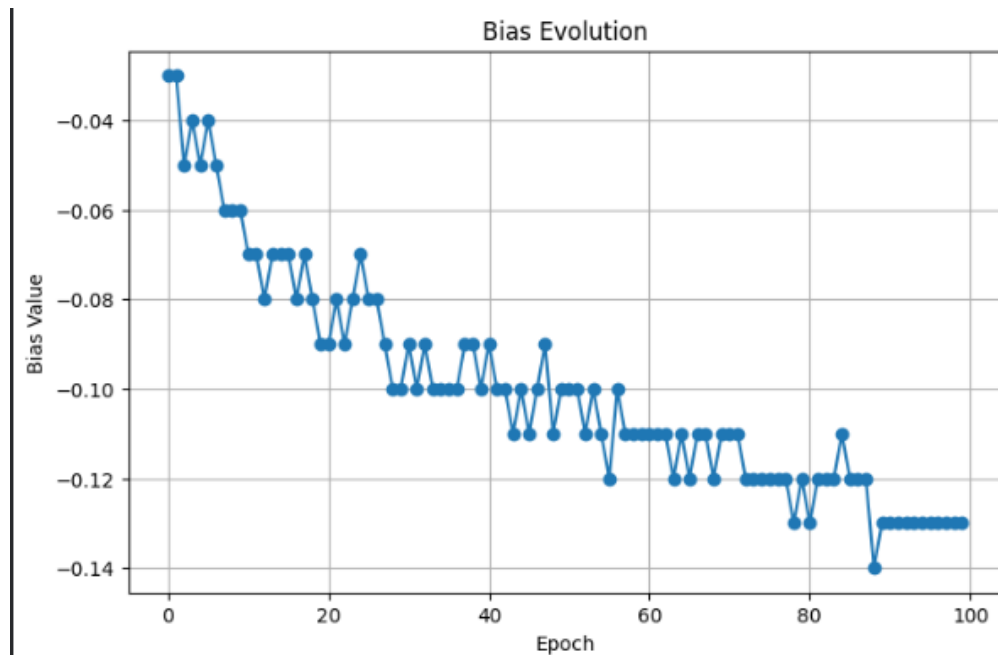
The number of misclassified samples decreases as the epochs increase, showing that the perceptron learns effectively during training. The training error stabilizes after many epochs, indicating convergence. The number of the misclassified samples decreases as the epochs increases, it shows that perceptron learnt effectively during training process. The training error stabilizes after several epochs, that indicates convergence.

6. Weight Evolution



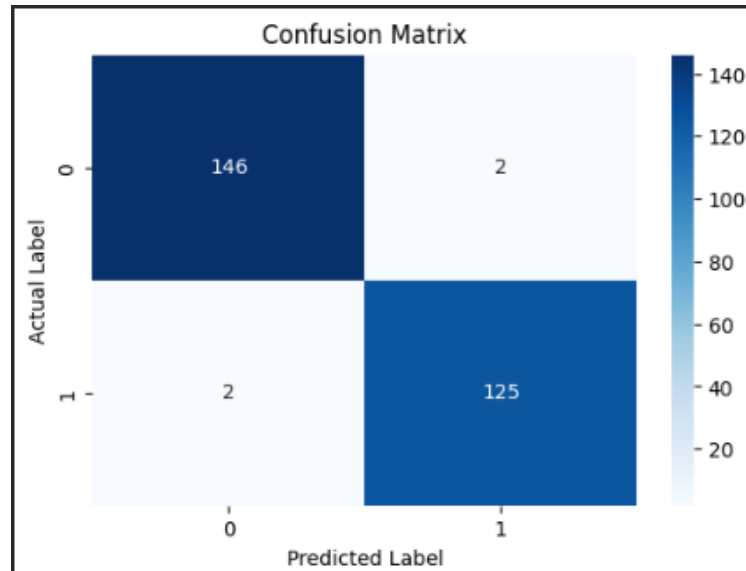
The weight values changes rapidly during the initial epochs and gradually after initial epochs stabilizes as training progresses. This indicates that the perceptron learns the optimal feature weights successfully.

7. Bias Evolution



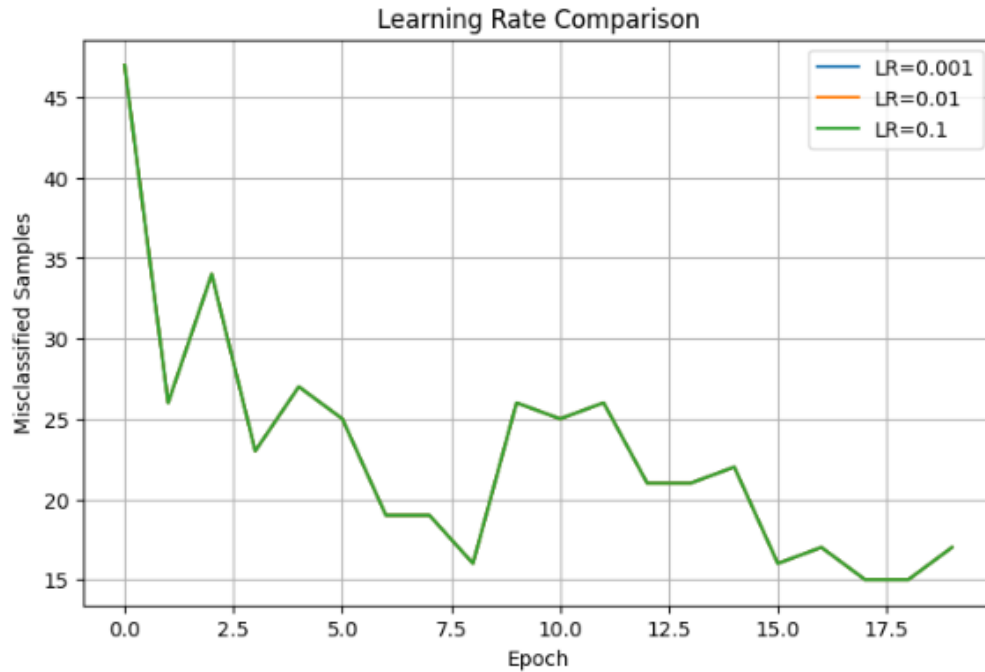
The bias gradually decreases during the training process and becomes stable near the final epochs. This suggests that the perceptron has converged to a stable decision boundary.

8. Confusion Matrix



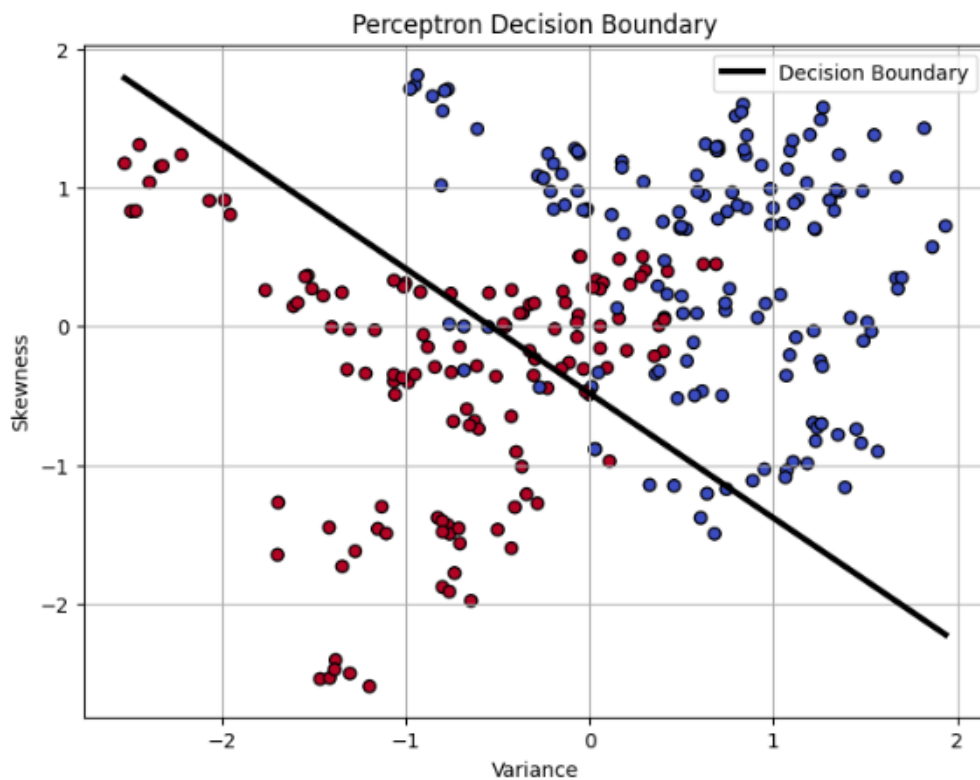
The confusion matrix shows that only 4 out of the 275 test samples were misclassified. The model achieved an accuracy of 98.55%, demonstrating excellent classification performance.

9. Learning Rate Comparison



The learning rate comparison shows that different learning rates exhibits similar convergence behaviour on the dataset. The final classification performance remains nearly identical because that the dataset is almost linearly seperable.

10. Decision Boundary



The decision boundary mostly separates the most of the samples into their respective classes with few less overlap. This confirms that the single layer perceptron learned an effective linear classifier for the selected features.

8. Performance Tables

Training Summary

Parameter	Value
Dataset Size	1372 Samples (1372×5)
Train/Test Split	80% Training (1097 samples), 20% Testing (275 samples)
Learning Rate	0.01
Epochs	100
Final Weights	[-0.2427, -0.2703, -0.2484, 0.0056]
Final Bias	-0.13
Accuracy	98.55%
Precision	98.43%
Recall	98.43%
F1-score	98.43%

Epoch-wise Learning

Epoch	Errors	Weight 1	Weight 2	Bias
1	47	-0.0725	-0.0649	-0.03
20	17	-0.1564	-0.1747	-0.09
40	19	-0.1903	-0.2030	-0.10
60	16	-0.2117	-0.2390	-0.11
80	16	-0.2365	-0.2637	-0.12
100	12	-0.2427	-0.2703	-0.13

9. Additional Tasks

1. Comparison of Step and Sigmoid Activation Functions

Step activation function produces only binary outputs (0 or 1) and it is non differentiable, making it not suitable for the gradient based optimisation. Whereas Sigmoid activation function produces continuous outputs between 0 and 1 and it is differentiable, allowing efficient backpropagation. So, modern deep learning models prefer sigmoid activation function or other differentiable activation functions than the step function.

2. Comparison with Scikit-learn's Perceptron

The perceptron implemented from scratch successfully classified the Banknote Authentication dataset with an accuracy of **98.55%**. Scikit-learn's **Perceptron** provides similar accuracy while offering optimized implementations, automatic parameter tuning, early stopping, and faster convergence. However, implementing the algorithm from scratch provides a better understanding of the perceptron learning process.

3. Learning Rate Comparison

This experiment was repeated using learning rates **0.001,0.01,**and **0.1**.All the three learning rates produced similar classification accuracy because the dataset is nearly linearly seperable.The primary difference is the magnitude of weight updates and convergence behaviour during the training.

4. Why a Single Layer Perceptron Cannot Solve the XOR Problem

The XOR problem is not linearly seperable ,meaning that the single straight decision boundary cannot correctly classify the all inputs.Since a single layer perceptron can only learn the linear decision boundaries,it could not able to solve the XOR problem.Multilayer neural network with hidden layers are required to learn the non-linear relationships.

5. Effect of Feature Normalization on Model Convergence

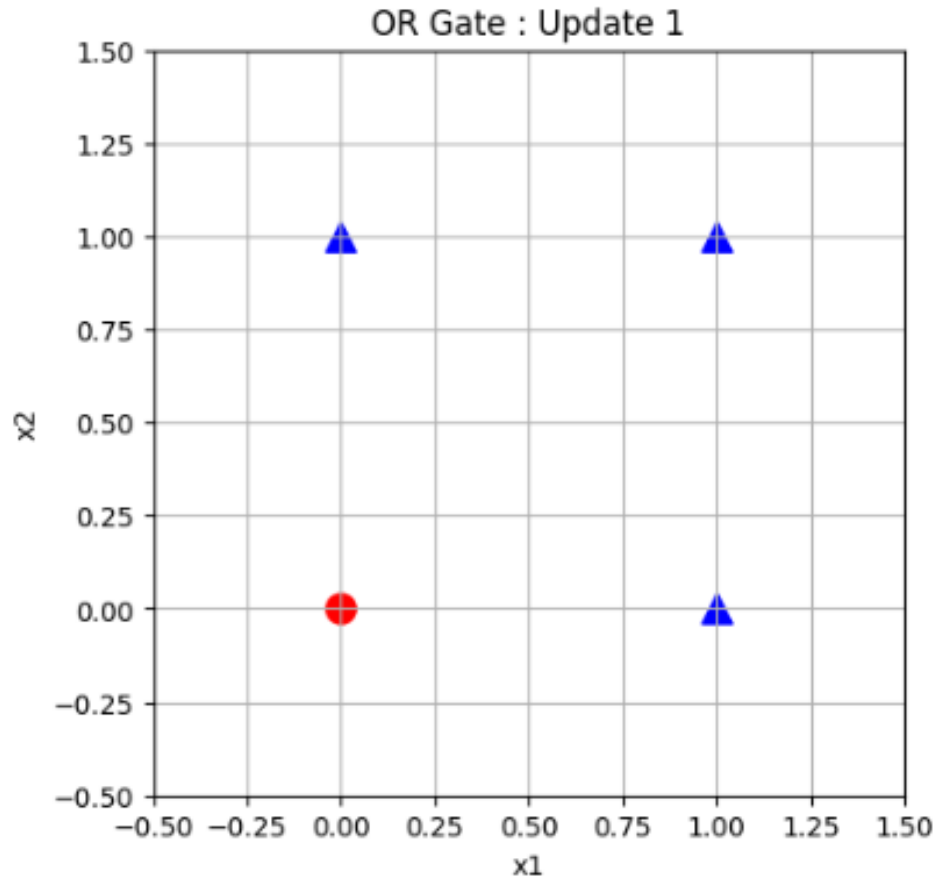
Feature normalization scales all the input features into a comparable range,preventing features with the larger numerical values from dominating the learning.In this experiment,normalization resulted in smoother weight updation,faster convergence,and improved the classification performances,which enables the perceptron to achieve an accuracy of **98.55%**

1.1 Implementation of Perceptron Learning Algorithm

Question:

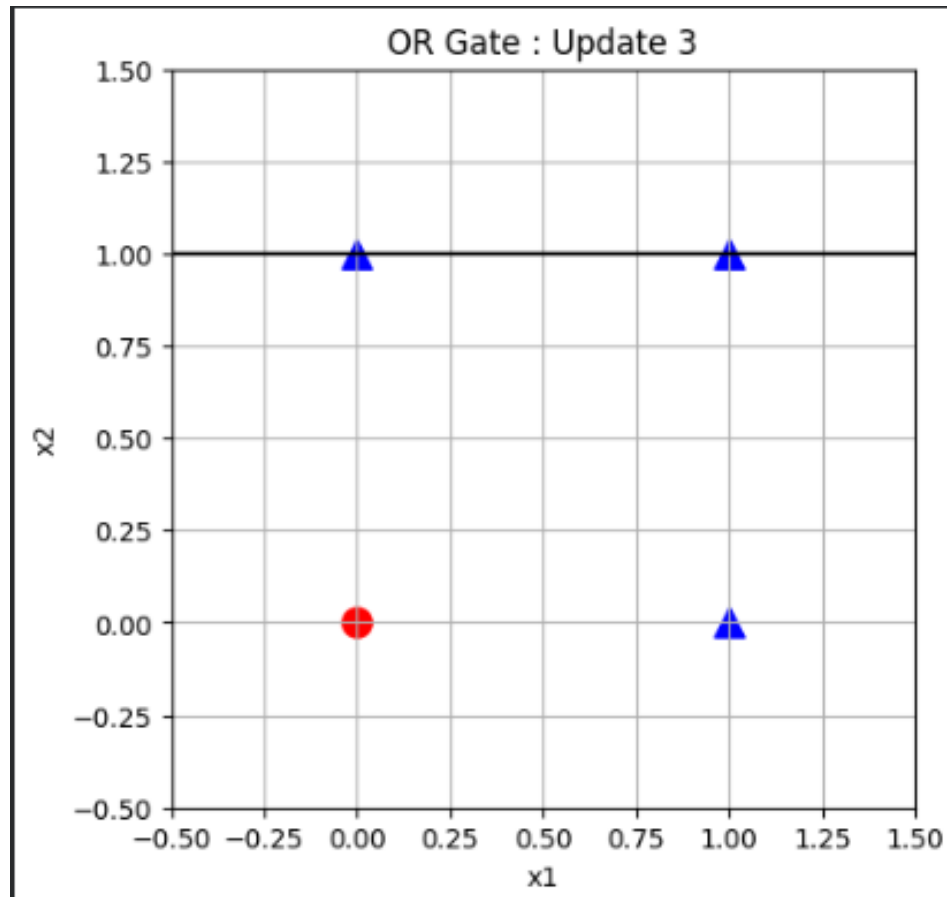
Code the perceptron learning algorithm for OR, NOT, AND, and XOR gates using Python. Show the weights after each update. Plot the decision boundary after each weight update using Python built in packages. Analyze the learning behavior for each logic gate. (K4)

1. OR Gate - Update 1



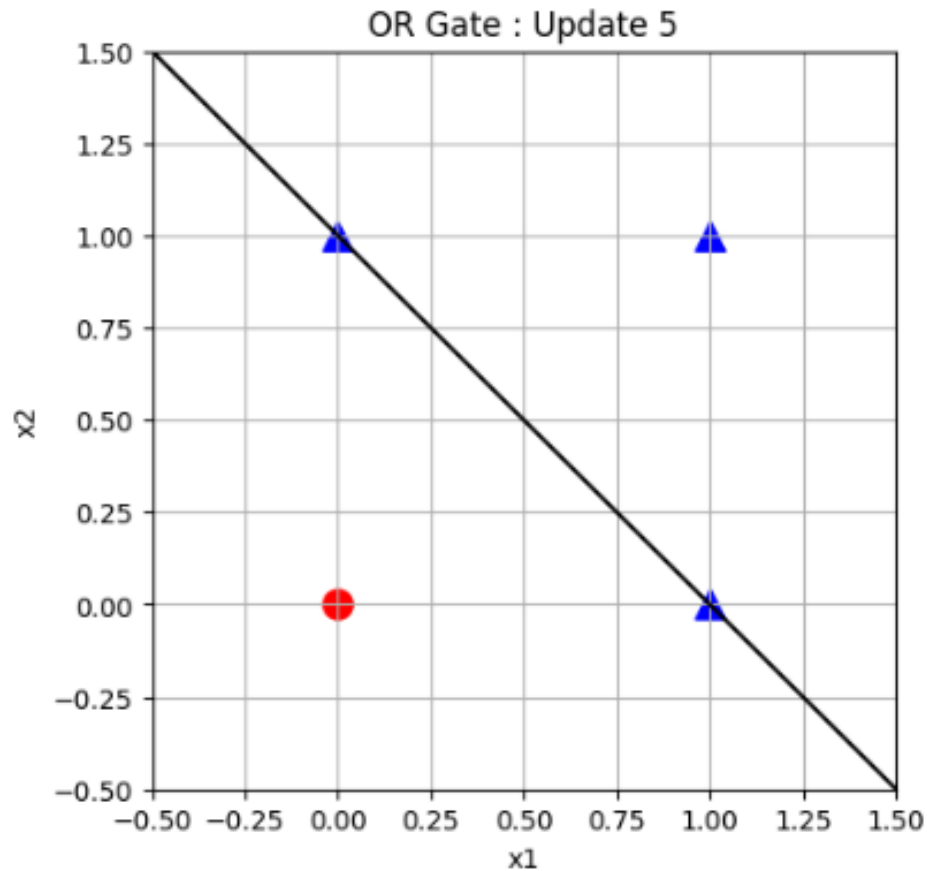
The initial decision boundary does not correctly separate the OR gate outputs since the perceptron has only completed its first update. Most samples are still misclassified because the weights are close to their initial values. This stage represents the beginning of the learning process before the model identifies the correct linear boundary.

2. OR Gate - Update 3



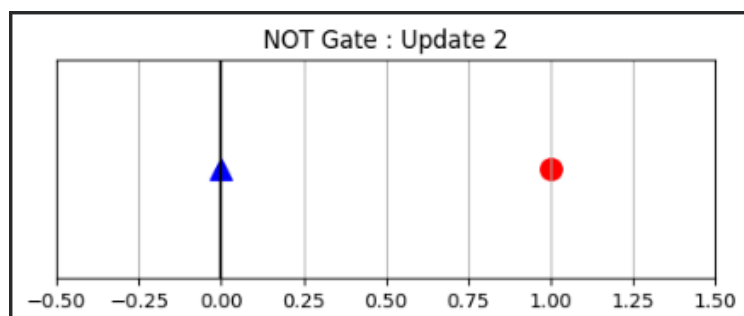
The decision boundary shifts noticeably after successive weight updates, reducing the number of misclassified samples. With weights approximately equal to $[0, 1]$ and a bias of -1 , the perceptron correctly classifies most of the training points. This indicates that the model is steadily learning the OR gate pattern.

3. OR Gate - Final Update



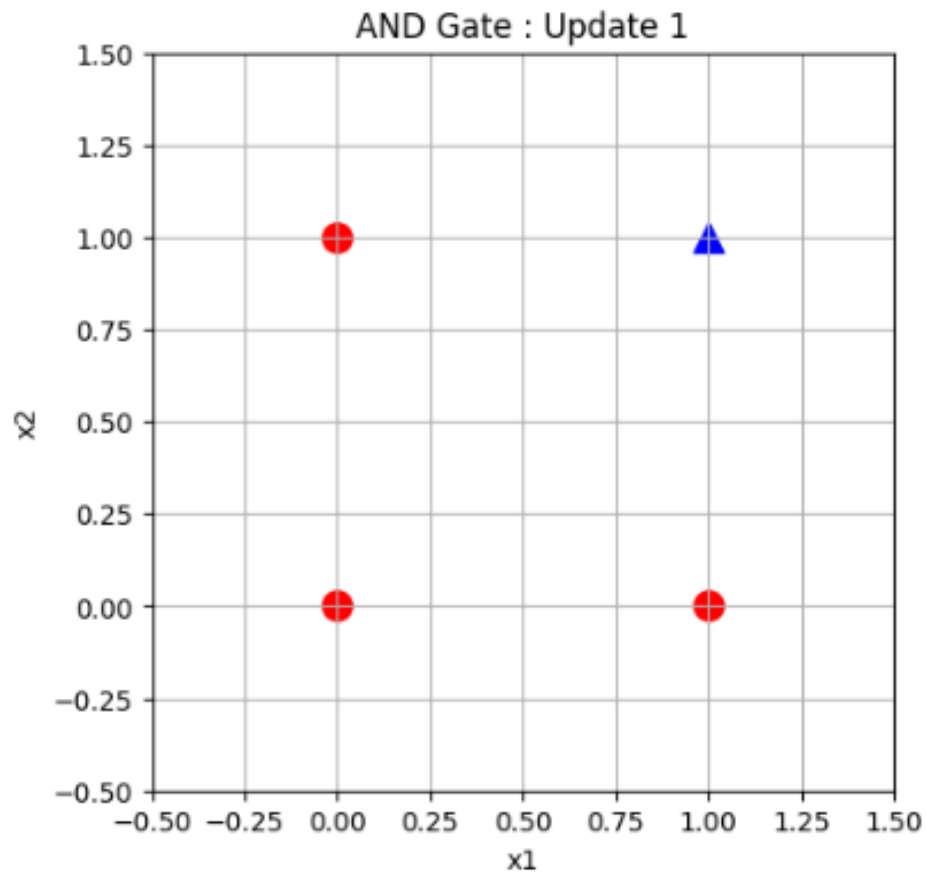
The final decision boundary perfectly separates the negative sample from the positive samples. The perceptron converges with the weights close to $[1, 1]$ and a bias of -1 , resulting in the correct classification of all four input combinations. This confirms that the OR gate is linearly separable.

4. NOT Gate - Final Decision Boundary



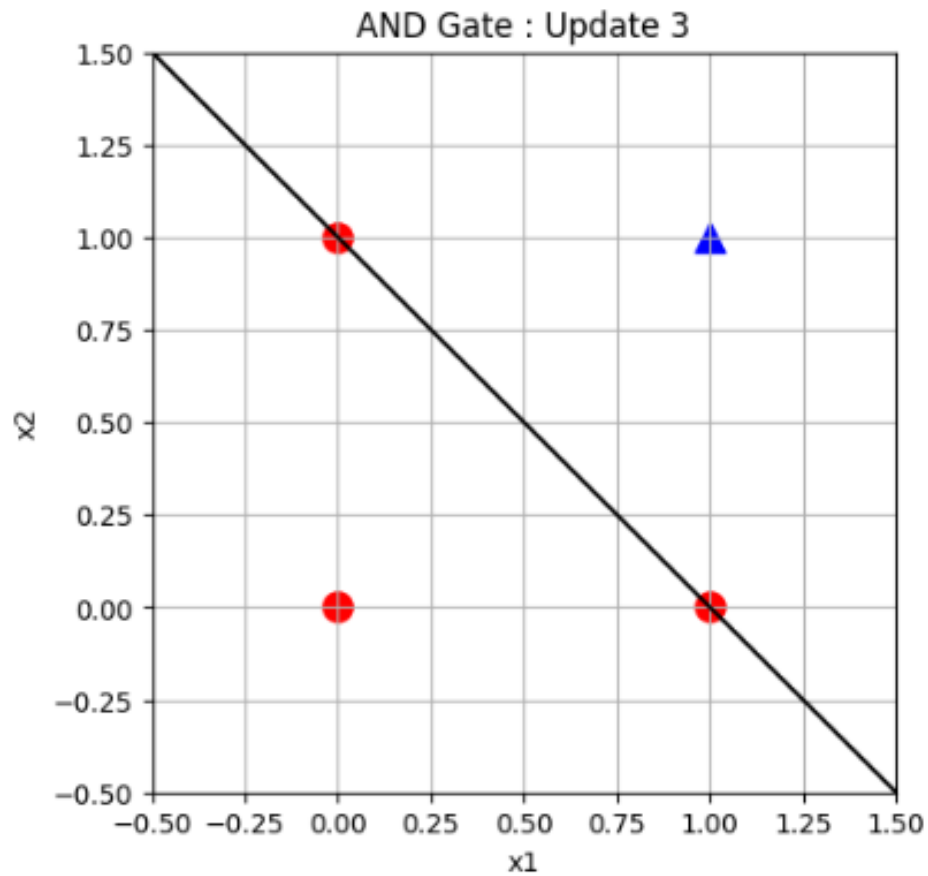
The figure shows a clear threshold separating the two possible input values of the NOT gate. Since only one feature is involved, the perceptron quickly learns the correct decision boundary with minimal weight updates. Both inputs are classified correctly, demonstrating successful convergence.

5. AND Gate - Update 1



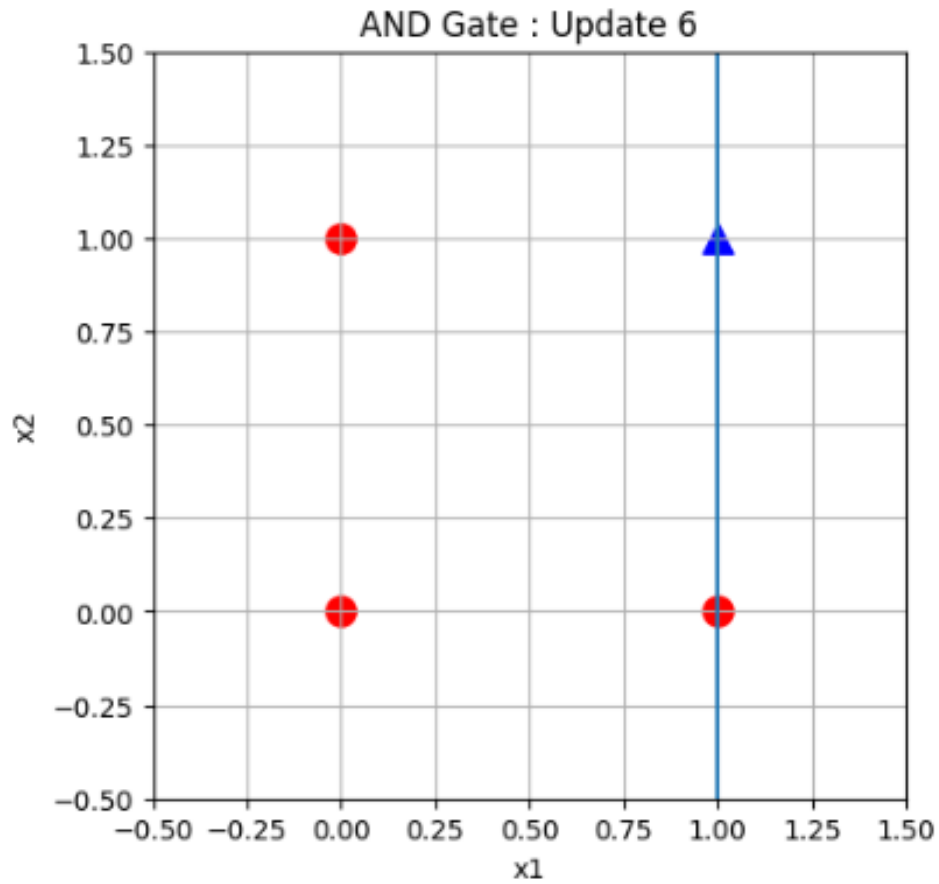
During the first iteration, the decision boundary is unable to isolate the positive sample (1,1) from the remaining negative samples. The perceptron has only begun adjusting its weights and bias, resulting in several incorrect classifications. Further learning is required before convergence can be achieved.

6. AND Gate - Update 3



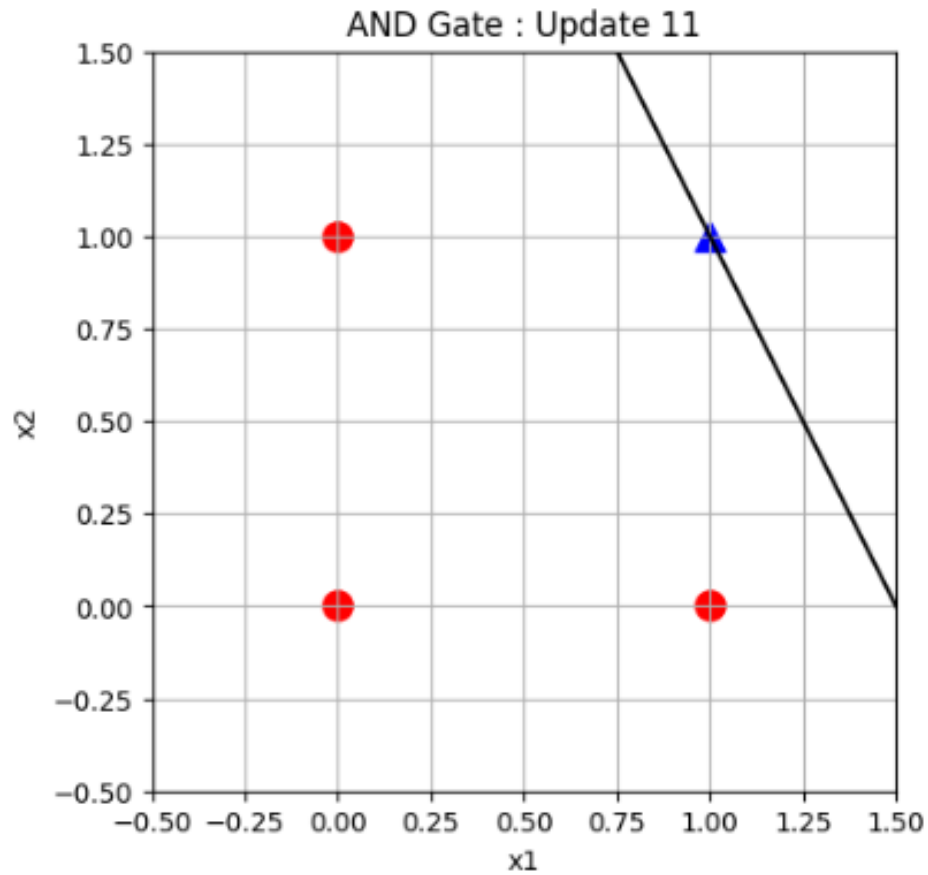
After several updates, the decision boundary moves closer to the desired position and classification accuracy improves. The updated weights and bias reduce the number of misclassified samples, showing that the perceptron is gradually learning the AND gate. However, the model has not yet reached its optimal solution.

7. AND Gate - Update 6



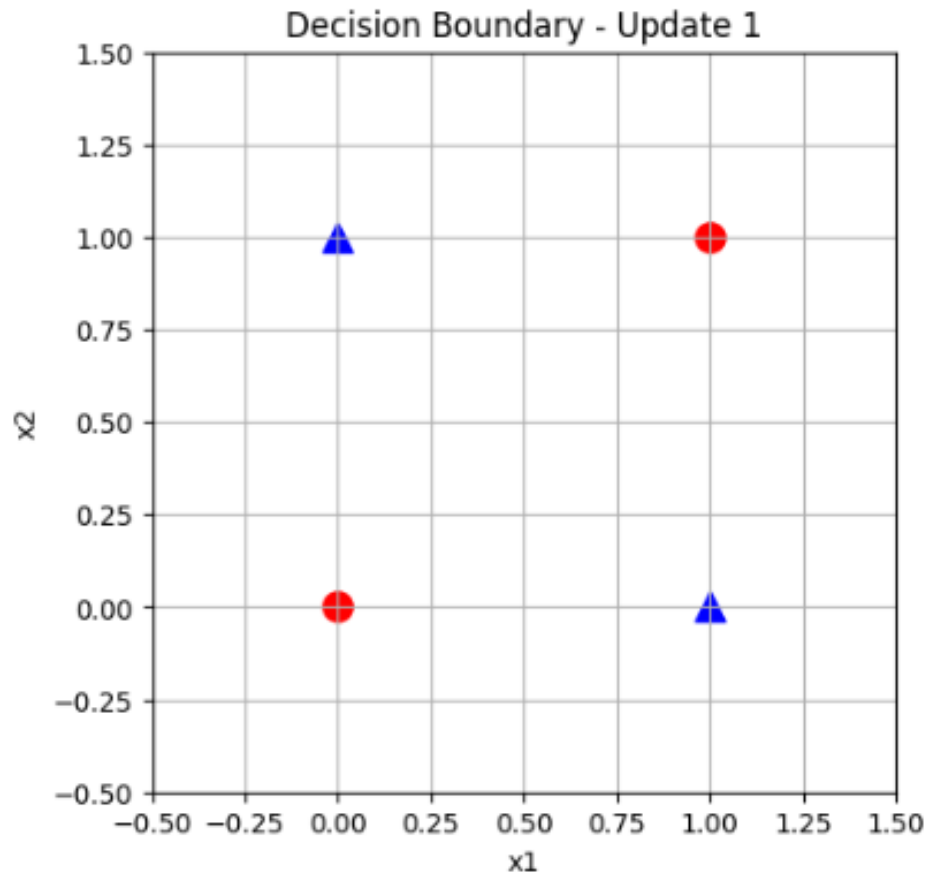
The decision boundary becomes more stable after additional training iterations and correctly separates most of the input combinations. Only minor adjustments remain before convergence is achieved. This demonstrates the gradual improvement of the perceptron through iterative weight updates.

8. AND Gate - Final Update



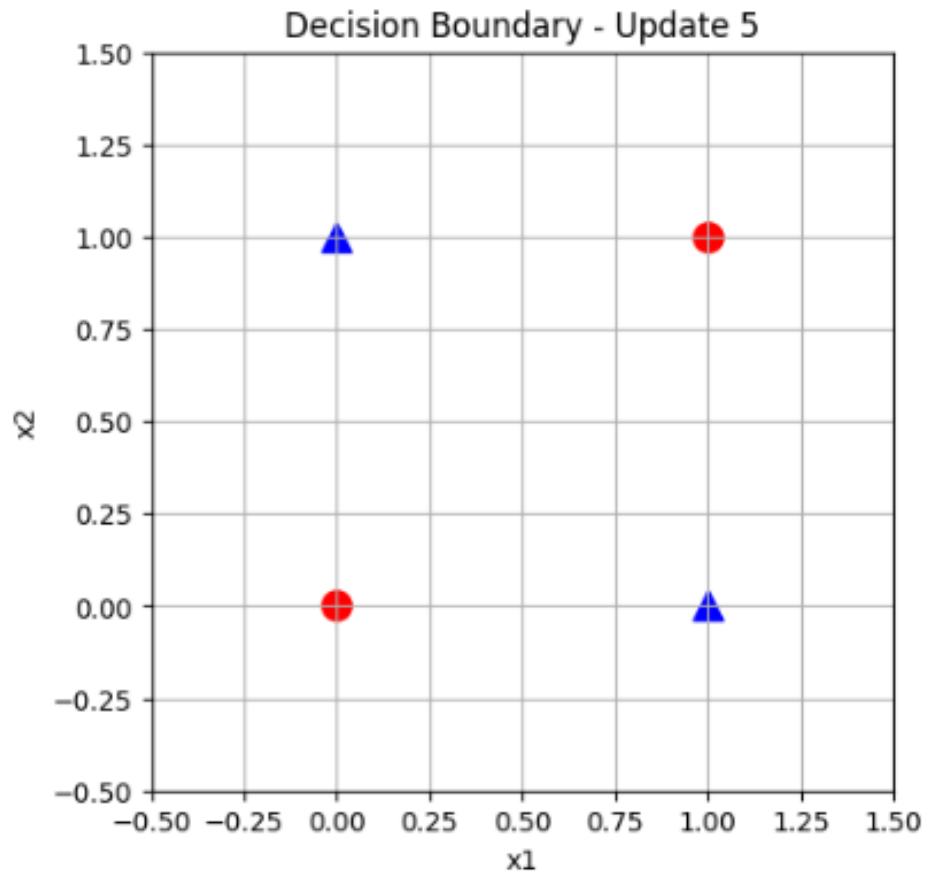
The perceptron successfully converges with final weights approximately equal to $[2, 1]$ and a bias of -3 . The learned decision boundary correctly isolates the only positive sample $(1, 1)$ while classifying all remaining inputs as negative. This confirms successful learning of the AND gate.

9. XOR Gate - Update 1



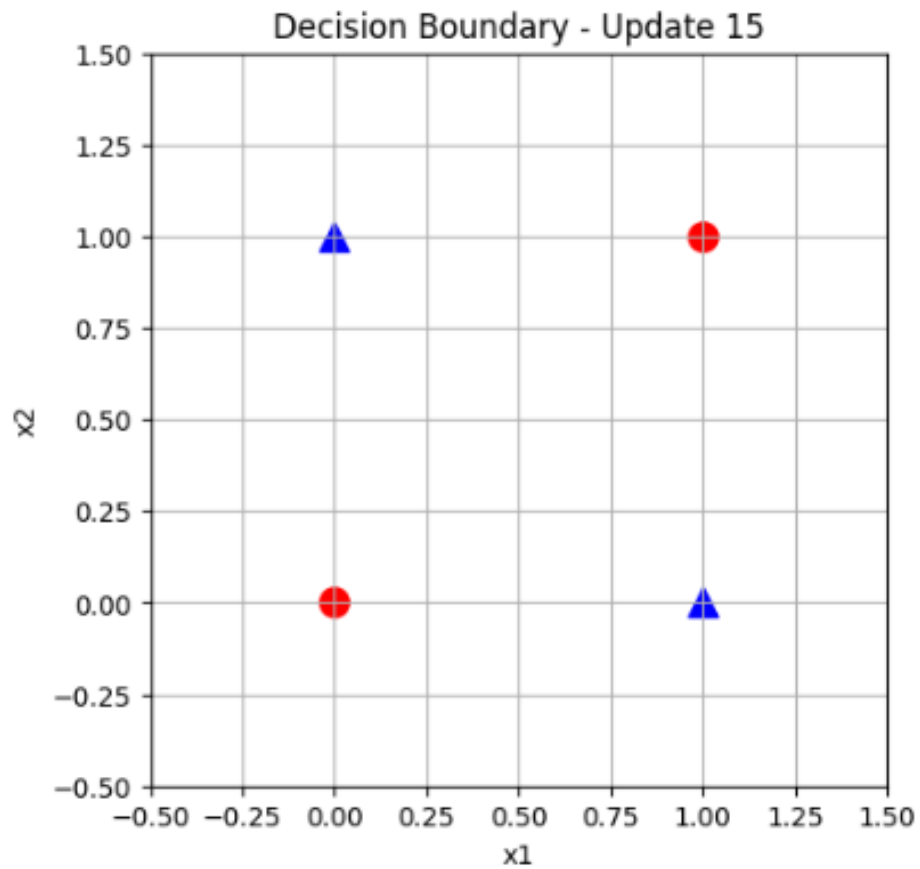
The first update produces only a slight change in the decision boundary because the weights remain close to their initial values. Several XOR samples are still incorrectly classified, indicating that the perceptron does not yet identified a suitable separating line. This is expected during the initial stage of training.

10. XOR Gate - Update 5



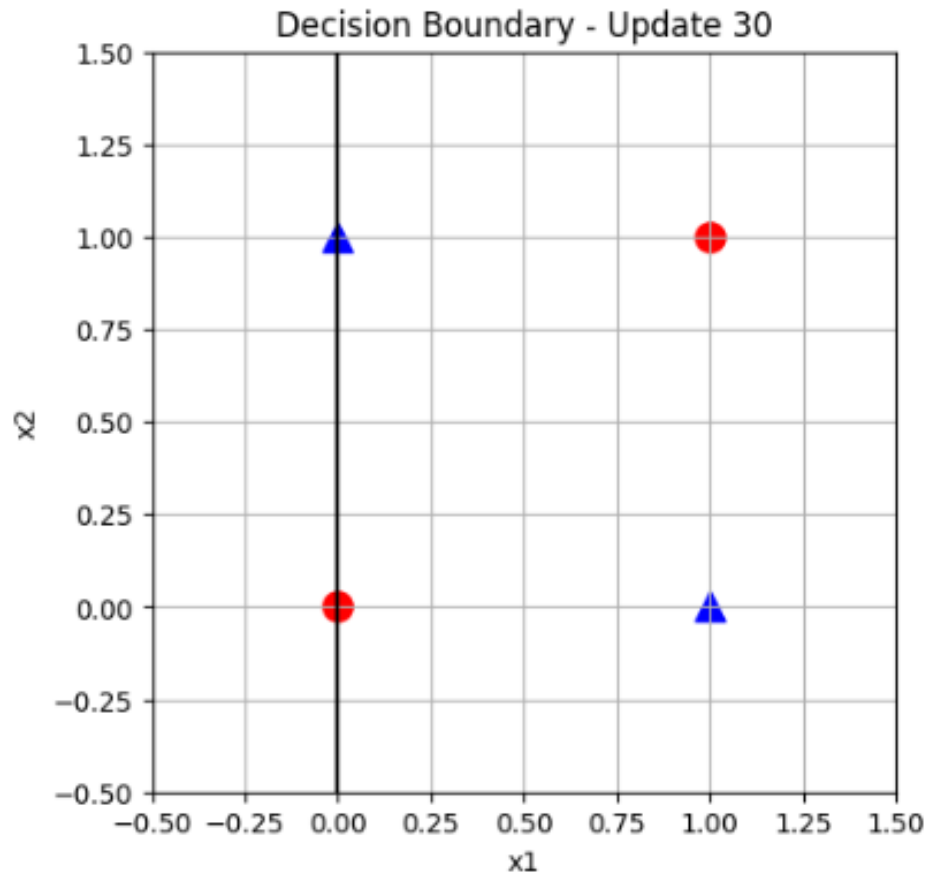
The decision boundary shifts after the multiple weight updates and correctly classifies some of the training samples. Although the weights become approximately $[0, 1]$ with a bias of 1, at least one XOR input remains misclassified. This shows that a single linear boundary is insufficient for solving the XOR problem.

11. XOR Gate - Update 15



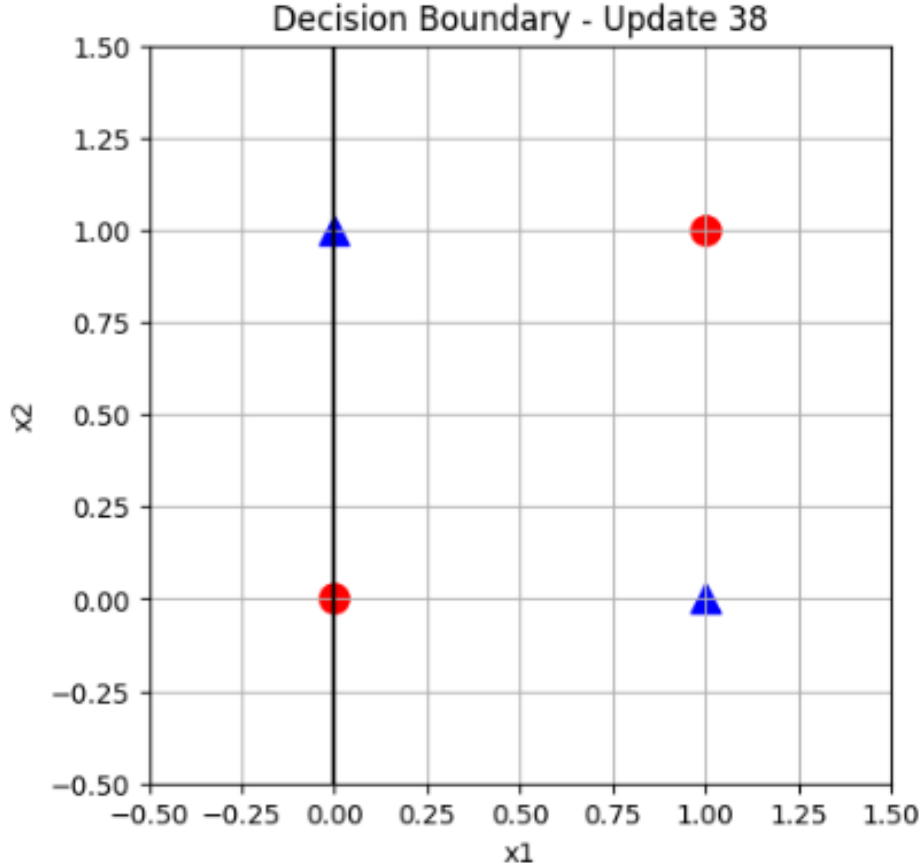
The decision boundary continues changing position, but instead of improving consistently, it starts revisiting earlier configurations. The weights become approximately $[-1, 0]$ with a bias of -1 , indicating that the learning process has entered a repetitive pattern. This behaviour suggests that convergence is unlikely.

12. XOR Gate - Update 30



The figure shows that the decision boundary has returned to a previously observed position instead of moving toward a stable solution. The weights again become approximately $[-1, 0]$ while the bias changes to 0. This repeated behaviour highlights the inability of a single-layer perceptron to separate XOR data.

13. XOR Gate - Final Update



The final figure confirms that the perceptron does not converge, as the decision boundary repeats an earlier configuration instead of stabilizing. The algorithm terminates with weights approximately equal to $[-1, 0]$ and a bias of 0, while some samples remain misclassified. This demonstrates that the XOR gate is not linearly separable and cannot be learned using a single-layer perceptron.

Overall Observation

The perceptron successfully learned the OR, NOT, and AND gates because each of these problems are linearly separable and it can be represented using single decision boundary. As training progressed, the decision boundaries gradually shifted toward their optimal positions until convergence was achieved. In contrast, the XOR gate repeatedly changed its decision boundary without reaching a stable solution, demonstrating the limitation of a single-layer perceptron for non-linearly separable problems. This observation highlights the necessity of multilayer neural networks with hidden layers to solve complex classification tasks.

10. Conclusion

This experiment successfully implemented single layer perceptron from scratch for binary classification using the BankNote Authentication dataset. The dataset explores, preprocessed through feature normalisation, and it is divided into training and testing sets before model training. The perceptron

effectively learned the decision boundary and achieved an accuracy of **98.55%** with high precision, recall, and F1-score. The training error decreases over epochs, weights and bias converges to stable values, demonstrating successful learning. The comparison of learning rates shown similar classification performance due to the nearly linearly separable nature of the dataset. Also the limitations of the perceptron such as XOR. overall the experiment provided a strong understanding of the perceptron learning, parameter updates, activation functions, model evaluation and the importance of the feature preprocessing in the machine learning.

11. References

1. F. Rosenblatt, "The Perceptron," *Psychological Review*, 1958.
2. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016.
3. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
4. S. Haykin, *Neural Networks and Learning Machines*, Pearson, 2009.
5. UCI Machine Learning Repository – Banknote Authentication Dataset.
6. Scikit-learn Documentation: Perceptron.